

OWASP Top 10 Web Risks

What OWASP Is

Open Worldwide Application Security Project. Every few years, security experts rank the most common & dangerous web app risks. It's the shared vocabulary of web security.

The 10 (2021 Edition)

Broken Access Control (A01) â€” users CAN view/modify data they shouldn't (IDOR example below)

Cryptographic Failures (A02) â€” weak hashing, hardcoded keys, HTTP data at rest

Injection (A03) â€” SQL, XSS, LDAP, OS commands

Insecure Design (A04) â€” trusting client input by design (trust boundaries)

Security Misconfiguration (A05) â€” default passwords, debug on, verbose errors

Vulnerable & Outdated Components (A06) â€” a library with a known CVE exploited

Identification & Authentication Failures (A07) â€” weak passwords, session loopholes

Software & Data Integrity Failures (A08) â€” updates/CI pipeline compromised, deserialization

Security Logging & Monitoring Failures (A09) â€” breaches unnoticed for months (the OWASP "sorry" of 2025: many breaches go unseen)

Server-Side Request Forgery (SSRF) (A10) â€” server fetches attacker-controlled URLs (hit cloud metadata!)

Read One Deep: IDOR (A01)

```
GET /api/orders/1001 â†’ your order
GET /api/orders/1002 â†’ SOMEONE ELSE's order?
```

Insecure Direct Object Reference: the server trusts the object ID in the URL/number without checking ownership. Canonical bug, enormous blast radius.

The fixed version: always check authorization server-side before returning ANY resource. Never trust IDs from the client.

Read Another Deep: Injection (A03)

SQL injection:

```
-- the buggy query:
SELECT * FROM users WHERE name = '<USER_INPUT>'
-- input: ' OR '1'='1'
SELECT * FROM users WHERE name = '' OR '1'='1'  -- ALL users!!
```

Fix: **parameterized queries / prepared statements** â€” never concatenate user input into SQL.

The Pattern Behind All 10

Ask of every feature: **is user input ever trusted to be honest?**

IDOR â†’ trusting IDs in URLs (an object ID is user input)

Injection â†’ trusting string content

SSRF â†’ trusting URLs the server follows

Access control â†’ trusting the cookie to mean "admin"

Defense in depth is architecture, not luck. Read the OWASP docs on your own â€” every one of these has a "Prevention Cheat Sheet."

[← Module start](#)

[SQL Injection & XSS Deep Dive →](#)